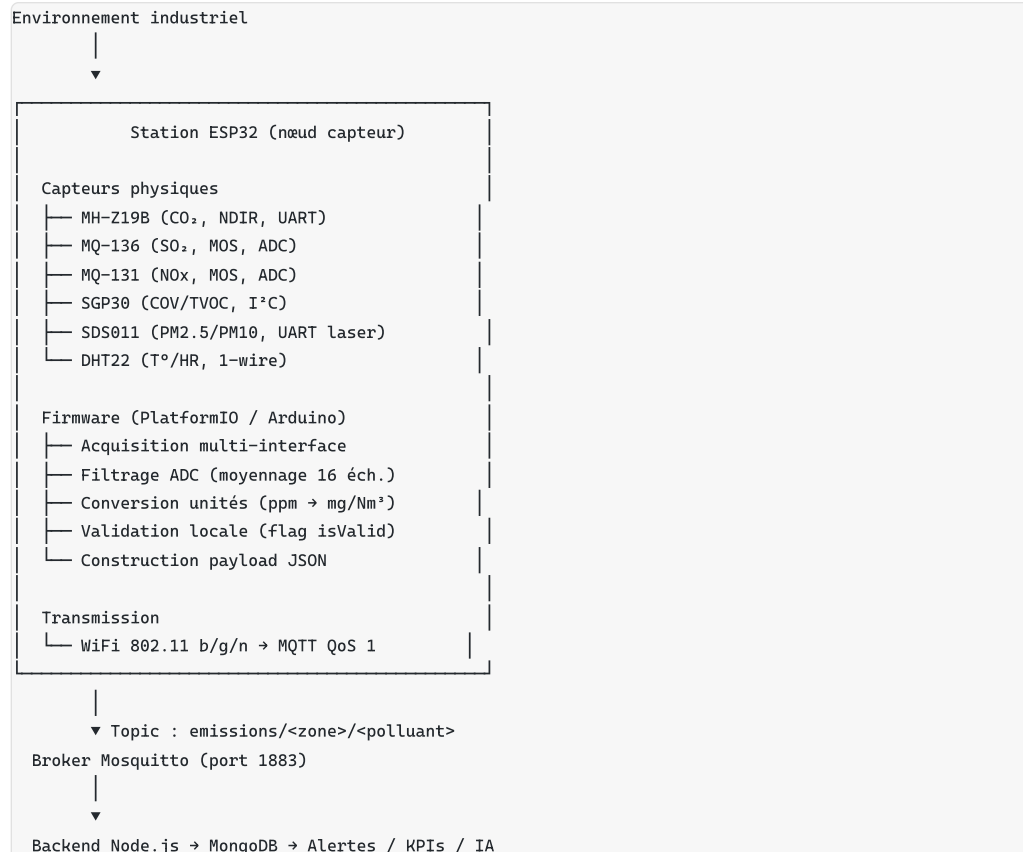


II. Couche IoT — Conception de la Station de Surveillance

II.1 Vue d'ensemble de l'architecture IoT

II.1.1 Position de la couche IoT dans le système global

La couche IoT constitue le premier maillon de la chaîne de surveillance. Elle est responsable de l'acquisition physique des concentrations de polluants, de leur traitement local, et de leur transmission vers le backend via le protocole MQTT. La chaîne fonctionnelle de bout en bout est la suivante :



Toute la logique de stockage, de calcul des KPIs et de génération d'alertes réglementaires est déportée sur le backend Node.js. La station ESP32 n'effectue qu'un **calcul de niveau indicatif local** (normal / warning / high / critical) destiné au diagnostic terrain, sans prétendre se substituer à l'évaluation réglementaire officielle.

II.2 Choix de la carte microcontrôleur

II.2.1 Critères de sélection

Le choix de la carte microcontrôleur est guidé par quatre contraintes simultanées :

1. **Connectivité WiFi native** pour publier directement sur le broker MQTT sans module réseau externe.
2. **Richesse des interfaces** : deux UART matériels pour MH-Z19B et SDS011, un bus I²C pour SGP30, deux canaux ADC pour les capteurs MOS MQ, une broche GPIO 1-wire pour DHT22.
3. **ADC 12 bits WiFi-safe** : les capteurs MQ imposent une lecture ADC pendant que le stack WiFi est actif — une contrainte qui élimine l'ADC2 de l'ESP32 (incompatible WiFi).
4. **Disponibilité locale et coût** : le projet cible une installation à Sfax, ce qui impose une disponibilité locale des composants.

II.2.2 Tableau comparatif

Critère	ESP32 DevKit V1 ✓	Arduino Mega 2560	Raspberry Pi Pico W
WiFi intégré	Oui — 802.11 b/g/n 2,4 GHz	Non (shield externe)	Oui — CYW43439
Fréquence CPU	240 MHz dual-core Xtensa LX6	16 MHz single-core AVR	133 MHz dual-core Cortex-M0+
RAM	520 KB SRAM	8 KB SRAM	264 KB SRAM
UART matériels	3 (Serial0/1/2)	4	2
ADC résolution	12 bits (ADC1 GPIO32–39, WiFi-safe)	10 bits	12 bits
Bus I ² C	Oui (SDA/SCL configurables)	Oui	Oui
Bibliothèque MQTT	PubSubClient (Arduino) — mature	Limitée, no WiFi natif	MicroPython / C SDK — moins mature
Environnement dev	PlatformIO / Arduino IDE	Arduino IDE	MicroPython / C SDK
Coût estimé (TN)	~15 DT	~25 DT	~18 DT
Verdict	Retenu	Écarté (RAM, WiFi)	Alternative — écosystème C++ moins mature

Conclusion. L'Arduino Mega est écarté pour son absence de WiFi natif et ses 8 KB de RAM, insuffisants pour maintenir simultanément les buffers UART des deux capteurs sériels (MH-Z19B, SDS011), le payload JSON (≈380 octets) et le stack TCP/IP. Le Raspberry Pi Pico W constitue une alternative valable pour les projets orientés MicroPython, mais son écosystème Arduino C++ pour les bibliothèques de capteurs industriels est moins mature. L'ESP32 répond à l'ensemble des contraintes et s'impose comme le choix optimal.

II.2.3 Architecture interne ESP32 exploitée

L'architecture dual-core du module WROOM-32 est mise à profit de façon naturelle par le framework Arduino/ESP-IDF :

- **Cœur 0** (protocolaire) : gestion du stack WiFi 802.11, connexion MQTT, maintien du keep-alive.
- **Cœur 1** (applicatif) : acquisition des capteurs, calcul de conversion, construction du payload JSON.

Cette séparation garantit que les reconnexions réseau (coupure WiFi, timeout MQTT) n'interrompent pas le cycle d'acquisition des capteurs.

II.3 Contraintes d'alimentation — Conception du circuit

II.3.1 Analyse des besoins en courant

Avant de concevoir la chaîne d'alimentation, il est nécessaire d'établir le bilan de courant de tous les composants de la station.

Composant	Tension nominale	Courant typique	Courant pic
ESP32 DevKit V1 (WiFi actif + CPU)	5 V → VIN	240 mA	500 mA
MH-Z19B (CO ₂ NDIR)	5 V	150 mA	150 mA
SDS011 (PM laser + ventilateur)	5 V	70 mA	200 mA (démarrage)
MQ-136 (filament chauffant SO ₂)	5 V	150 mA	200 mA
MQ-131 (filament chauffant NOx)	5 V	150 mA	200 mA
SGP30 (COV I ² C)	3,3 V	49 mA	49 mA
DHT22 (T°/HR 1-wire)	3,3 V	1,5 mA	2,5 mA
Total estimé (régime permanent)		≈ 810 mA	≈ 1 300 mA

Un bloc d'alimentation **5 V / 3 A** offre une marge de sécurité suffisante (facteur ×2,3 sur le régime permanent, facteur ×2,3 sur le courant pic estimé).

II.3.2 Choix architectural : alimentation directe 5 V sans régulateur externe

Justification de l'absence du LM2596

La conception initiale envisageait un régulateur buck LM2596 pour produire le rail 3,3 V destiné au SGP30 et au DHT22. Ce composant est **supprimé** après analyse, pour la raison suivante : l'ESP32 DevKit V1 intègre sur son PCB un **régulateur LDO AMS1117-3.3** entre la broche VIN et le rail 3,3 V interne. Ce régulateur est directement accessible depuis la broche **3V3** de la carte, et sa capacité de courant est de **800 mA** selon la fiche technique.

Le bilan de courant sur le rail 3,3 V se limite à :

Consommateur 3,3 V	Courant
SGP30	49 mA
DHT22	1,5 mA
Logique interne ESP32 (processeur, RAM, périphériques)	~80 mA
Total sur AMS1117	≈ 130 mA

130 mA est très en dessous du seuil de 800 mA du LDO intégré. **Le LM2596 est donc redondant** pour cette configuration de capteurs. L'ajouter représenterait un coût, une surface de câblage et une complexité supplémentaires sans apport réel.

Discussion des limites de ce choix

Ce choix est valide pour le présent prototype mais mérite deux observations :

1. Le LDO AMS1117 est un régulateur linéaire. Il dissipe la différence de tension en chaleur :

$$P_{\text{dissipée}} = (V_{\text{IN}} - V_{\text{OUT}}) \times I_{\text{total}} = (5 - 3,3) \times 0,13 \approx 221 \text{ mW}$$

$P_{\text{dissipée}} = (V_{\text{IN}} - V_{\text{OUT}}) \times I_{\text{total}} = (5 - 3,3) \times 0,13 \approx 221 \text{ mW}$ 221 mW est tout à fait gérable thermiquement sur la petite puce de l'AMS1117 (résistance thermique ≈ 180 °C/W → élévation de température ≈ 40 °C au-dessus de l'ambient). Pour la charge actuelle, ce n'est pas un problème. En revanche, si le système était étendu avec des capteurs 3,3 V supplémentaires consommant plusieurs centaines de mA (ex. écran couleur TFT, module SD), le bilan thermique du LDO devrait être recalculé et un buck externe redeviendrait pertinent. **2. La qualité du rail 3,3 V dépend du bloc secteur.** Un bloc à découpage de bonne qualité produit un 5 V propre, et l'AMS1117 filtre efficacement le résidu de ripple basse fréquence. Les condensateurs de découplage (100 nF céramique au plus près de chaque IC) restent néanmoins **indispensables** pour filtrer les transitoires haute fréquence que le LDO ne peut pas atténuer. **Conclusion :** la suppression du LM2596 est un choix d'ingénierie adapté aux contraintes du projet : charge 3,3 V légère, prototype sur breadboard, contrainte budgétaire. Le régulateur intégré de l'ESP32 est suffisant. ### II.3.3 Architecture de la chaîne d'alimentation retenue La chaîne d'alimentation se réduit à **deux niveaux** : le bloc secteur fournit le 5 V, l'ESP32 produit le 3,3 V via son LDO interne. "" 220 V AC (secteur industriel) | ▼ Bloc alimentation à découpage 5V / 3A (USB / Jack DC) | | [1] Interrupteur On/Off | [2] Fusible 1A (protection surintensité) | [3] Diode 1N4007 (protection inversion de polarité) | [4] Condensateur 100 µF électrolytique (filtrage basse fréquence) | ▼ 5 V DC stabilisé | | | ▼ ESP32 DevKit V1 (VIN 5V) 5V bus direct (capteurs 5V) LDO interne AMS1117-3.3 → 3V3 (broche) | MH-Z19B (5V / 150mA) + 100nF céramique par

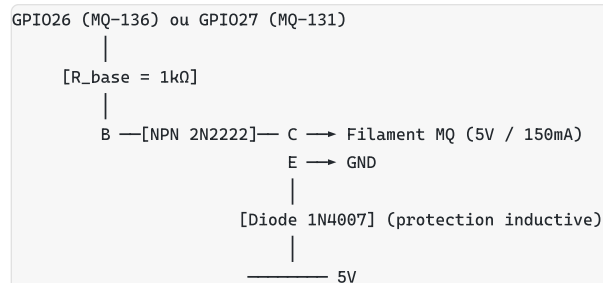
IC sur bus 3,3V | MQ-136 chauffage (5V / 150mA) | MQ-131 chauffage (5V / 150mA) ▼ | SDS011 (5V / 70mA) 3,3V (broche 3V3 ESP32) | SGP30 (I²C SDA/SCL + pull-up 4,7kΩ) | DHT22 (DATA GPIO4 + pull-up 10kΩ) ***Note sur les GPIO.** Bien que l'ESP32 soit alimenté en 5 V via VIN, ses broches GPIO sont en logique **3,3 V uniquement**. Tous les signaux provenant de capteurs alimentés en 5 V (MH-Z19B, SDS011) doivent passer par un **convertisseur de niveau bidirectionnel** avant les GPIO UART. ### II.3.4 Protection de la ligne d'alimentation **Fusible 1A (protection contre les surintensités).** Placé en série sur la ligne positive 5 V après le bloc secteur, il protège l'ensemble du circuit contre un court-circuit. La valeur de 1 A est choisie légèrement supérieure au courant de régime permanent estimé (~810 mA) mais inférieure au courant pic (1,3 A) pour déclencher rapidement en cas de défaut. **Diode 1N4007 (protection contre l'inversion de polarité).** Placée en série avec le fusible, elle bloque tout courant inverse qui résulterait d'une inversion du connecteur d'alimentation — une protection critique sur des bancs de prototypage où les connecteurs Jack DC peuvent être branchés dans les deux sens. **Interrupteur On/Off.** Placé avant le fusible sur la ligne AC ou DC d'entrée, il permet une coupure propre de l'alimentation sans débrancher le connecteur secteur. --- ## II.4 Sélection des capteurs — Conception des interfaces ### II.4.1 MH-Z19B — Dioxyde de carbone (CO₂, NDIR) **Principe physique.** La technologie NDIR (Non-Dispersive InfraRed) est la seule méthode portable capable d'identifier spécifiquement le CO₂ parmi les mélanges gazeux industriels. Une source infrarouge éclaire la cellule de mesure ; les molécules de CO₂ absorbent spécifiquement à 4,26 μm. La concentration est déduite de l'atténuation mesurée par le détecteur. Cette méthode est insensible à l'humidité et aux gaz interférents. **Interface électronique.** Le MH-Z19B communique via UART 9600 bauds (Serial2 de l'ESP32). Ses niveaux logiques sont en 5 V alors que les GPIO de l'ESP32 tolèrent 3,3 V maximum. Un **convertisseur de niveau bidirectionnel** est donc intercalé sur les lignes TX et RX. MH-Z19B (5V) Level Shifter ESP32 (3,3V) TX (5V) → LV HV → GPIO16 (RX Serial2) RX (5V) ← LV HV ← GPIO17 (TX Serial2) 5V → HV 3.3V → LV ***Alimentation.** 5 V via le bus direct, courant pic 150 mA. La fonction ABC (Automatic Baseline Correction) compense la dérive à long terme sans intervention manuelle. ### II.4.2 MQ-136 et MQ-131 — SO₂ et NO_x (capteurs MOS) **Principe physique (oxyde métallique).** Les capteurs MQ de la série Hanwei/Winsen utilisent une couche de SnO₂ (dioxyde d'étain) chauffée à ~200 °C par un filament résistif intégré. En présence du gaz cible, la résistance de surface R_s diminue selon une loi log-linéaire. La mesure de R_s, comparée à la résistance en air propre R₀, donne la concentration via la courbe de sensibilité du datasheet. **Pont diviseur résistif et interface ADC.** La sortie du capteur MQ est une tension entre 0 et 5 V, produite par le pont formé de la résistance capteur R_s et d'une résistance de charge R_L. L'ADC de l'ESP32 est limité à 3,3 V : un diviseur résistif ramène cette tension à une valeur compatible. Conception du circuit d'interface MQ : 5V | [R_{sensor}(MQ)] ← résistance variable avec la concentration | [R₁ = 10kΩ] → GND → V_{out_MQ} (0–5V) | [R_L = 10kΩ] ← résistance de charge (fixe) | GND V_{out_MQ} — [R_{up} = 10kΩ] — [R_{down} = 20kΩ] — GND | V_{out_ADC} (≤ 3,3V) → GPIO33 ou GPIO34 ***Calcul du ratio de division : **

$$V_{out_ADC} = V_{out_MQ} \times \frac{R_{down}}{R_{up} + R_{down}} = V_{out_MQ} \times \frac{20}{30} = V_{out_MQ} \times 0,667$$

$$V_{out_ADC} = V_{out_MQ} \times R_{up} + R_{down} R_{down} = V_{out_MQ} \times 3020 = V_{out_MQ} \times 0,667$$

Pour V_{out_MQ} V_{out_MQ} = 5 V maximum : V_{out_ADC} V_{out_ADC} = 3,33 V ≈ 3,3 V ✓

Chauffage MQ via transistor NPN 2N2222. Le filament chauffant des capteurs MQ consomme 150 à 200 mA sous 5 V. Ce courant dépasse largement la capacité de sortie d'un GPIO ESP32 (max 40 mA). Un transistor NPN 2N2222 est utilisé en commutation saturée pour piloter le chauffage :



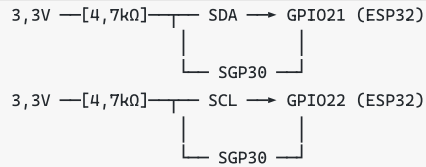
La résistance de base de 1 kΩ limite le courant de commande à $I_B = (3,3 - 0,7)/1000 \approx 2,6$ mA, suffisant pour saturer le 2N2222 (gain h_{FE} ≈ 100, donc I_C supporté ≈ 260 mA > 200 mA chauffage). La diode 1N4007 protège le transistor contre les surtensions induites par l'induction parasite du filament résistif.

Contrainte ADC WiFi. Lors de l'utilisation du WiFi, l'ADC2 de l'ESP32 est non fonctionnel (conflit avec le stack RF). Les deux capteurs MQ sont donc connectés sur des broches de l'**ADC1** uniquement (GPIO32–39) : GPIO33 pour le MQ-136 et GPIO34 pour le MQ-131. GPIO34 est en mode **input-only** (absence de résistance pull-up interne) — ce qui est parfaitement adapté à une lecture ADC pure.

II.4.3 SGP30 — Composés organiques volatils (COV, I²C)

Principe physique. Le SGP30 (Sensirion) est un capteur TVOC (Total Volatile Organic Compounds) à oxyde métallique chauffé. La mesure repose sur la variation de résistance d'une couche MOX en présence de COV. Le SGP30 intègre un algorithme de compensation d'humidité et produit un index TVOC en ppb et une estimation eCO₂ en ppm.

Interface I²C. Le SGP30 opère exclusivement en 3,3 V et communique via I²C à l'adresse fixe 0x58. Le bus I²C est partagé avec l'afficheur OLED SSD1306 (optionnel). Les **résistances de tirage 4,7 kΩ** entre SDA/SCL et 3,3 V sont obligatoires pour la stabilité du bus en mode Open-Drain.



II.4.4 SDS011 — Particules PM2.5 et PM10 (diffusion laser)

Principe physique. Le SDS011 (Nova Fitness) utilise un laser 650 nm pour compter les particules en suspension dans un flux d'air aspiré par un ventilateur intégré. La lumière diffusée par les particules est analysée par un photorécepteur pour déduire les concentrations PM2.5 et PM10 en µg/m³.

Mode query (acquisition à la demande). Le SDS011 est configuré en **mode query** dans le firmware : l'ESP32 envoie une commande d'acquisition, et le SDS011 répond avec une trame de 10 octets contenant PM2.5 et PM10. Ce mode évite le flux continu de données (mode actif par défaut) qui surchargerait le buffer UART et produirait des lectures parasites.

Interface. Identique au MH-Z19B : UART 9600 bauds + convertisseur de niveau bidirectionnel. Le SDS011 est assigné sur Serial1 (GPIO18/19).

II.4.5 DHT22 — Température et Humidité relative

Principe physique. Le DHT22 utilise un protocole 1-wire propriétaire sur un seul fil de données. La mesure de température est basée sur un capteur NTC et la mesure d'humidité sur un condensateur polymère. La précision est de ±0,5 °C et ±2–5 %RH.

Interface. Le GPIO4 est retenu pour sa compatibilité avec le protocole 1-wire (broche sans fonction de démarrage critique sur l'ESP32). Une résistance pull-up de **10 kΩ** entre la ligne DATA et 3,3 V est requise par le protocole.

II.5 Filtrage et traitement du signal

II.5.1 Moyennage ADC (capteurs MQ)

La sortie analogique des capteurs MQ-136 et MQ-131 est sujette à un bruit haute fréquence causé par le bruit thermique des résistances du pont diviseur, les perturbations électromagnétiques liées au filament chauffant (champ magnétique parasite ~150–200 mA), et le ripple résiduel du bloc à découpage 5 V. Pour améliorer le rapport signal/bruit, le firmware effectue un **moyennage sur 16 échantillons ADC** avec un délai de 5 ms entre chaque lecture :

Filtrage ADC – conception du moyennage :

Pour chaque lecture MQ :

```
Σ = 0
Pour i = 1 à 16 :
    Σ += analogRead(pin)
    delay(5 ms)
V_adc = (Σ / 16) × (3,3 / 4095)
```

Durée totale par lecture : 16 × 5 ms = 80 ms

Ce moyennage réduit le bruit blanc d'un facteur $\sqrt{16} = 4$

$\sqrt{16} = 4$, améliorant la résolution effective de 2 bits (de 12 à 14 bits équivalents).

II.5.2 Validation des plages ADC

Avant tout calcul de concentration, chaque tension ADC lue est vérifiée dans une plage physiquement plausible :

Condition	Signification	Action
$V_{ADC} < 0,05V$ $ADC < 0,05 V$	Capteur déconnecté ou court-circuit RS	<code>valid = false</code>
$V_{ADC} > 3,20V$ $ADC > 3,20 V$	Diviseur résistif absent ou saturé	<code>valid = false</code>
$0,05 \leq V_{ADC} \leq 3,20$ $0,05 \leq VADC \leq 3,20 V$	Lecture plausible	Calcul de concentration

II.5.3 Algorithme de conversion MQ (Rs/R0 → ppm → mg/Nm³)

La conversion de la tension ADC en concentration de polluant suit une chaîne de calcul documentée dans le firmware :

Étape 1 : Reconstituer la tension réelle au pont MQ $V_{MQ} = V_{ADC} / 0,6667$ (annule le diviseur 10k/20k)
Étape 2 : Calculer la résistance capteur Rs $R_s = R_L \times (V_{cc} / V_{MQ} - 1)$ (Rs en kΩ, RL = 10 kΩ, Vcc = 5V)
Étape 3 : Calculer le ratio de sensibilité $ratio = R_s / R_0$ (R0 = résistance en air propre)
Étape 4 : Appliquer la courbe log-linéaire du datasheet $ppm = A \times ratio^B$ MQ-136 (SO₂) : A = 36,7 B = -3,536 MQ-131 (NOx) : A = 23,943 B = -1,11
Étape 5 : Convertir en mg/Nm³ SO₂ : $mg/Nm^3 = ppm \times 2,86$ (facteur molaire SO₂, M = 64 g/mol) NOx : $mg/Nm^3 = ppm \times 2,05$ (facteur molaire NO₂, M = 46 g/mol)

Les facteurs de conversion molaires sont calculés à conditions normales (0°C, 101,325 kPa) : $1 ppm = M/22,4 mg/Nm^3$
 $1 ppm = M/22,4 mg/Nm^3$.

II.6 Affectation des GPIO — Résumé de conception

ESP32 DevKit V1 – Tableau d'affectation GPIO

GPIO 4	← DHT22 DATA	(pull-up 10 kΩ → 3,3V)
GPIO 16	← MH-Z19B TX	(UART RX, Serial2)
GPIO 17	→ MH-Z19B RX	(UART TX, Serial2)
GPIO 18	← SDS011 TX	(UART RX, Serial1)
GPIO 19	→ SDS011 RX	(UART TX, Serial1)
GPIO 21	↔ I²C SDA	(SGP30, pull-up 4,7kΩ)
GPIO 22	↔ I²C SCL	(SGP30, pull-up 4,7kΩ)
GPIO 26	→ MQ-136 Chauffage	(NPN 2N2222, R_base 1kΩ)
GPIO 27	→ MQ-131 Chauffage	(NPN 2N2222, R_base 1kΩ)
GPIO 33	← MQ-136 Vout	(ADC1 CH5, div. 10k/20k)
GPIO 34	← MQ-131 Vout	(ADC1 CH6, input-only)
VIN (5V)	← Alimentation principale (bus 5V)	
GND	← Masse commune à tous les modules	

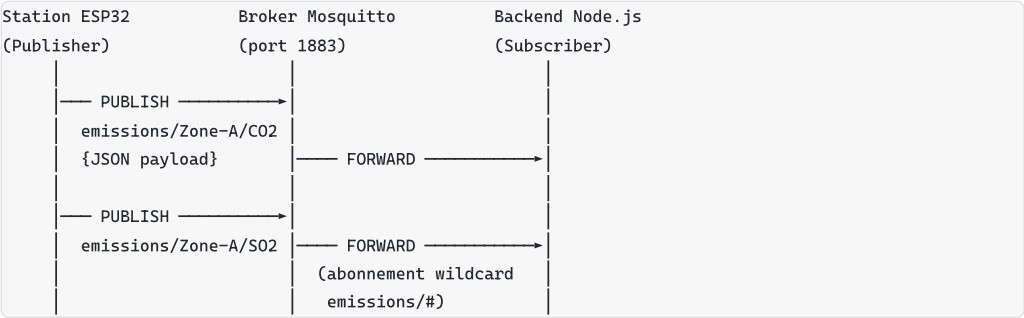
Règles critiques de câblage :

- ADC1 uniquement (GPIO32-39) – ADC2 incompatible WiFi actif
- GPIO34-39 : input-only, pas de pull-up interne disponible
- Masse GND commune 5V et 3,3V – référence ADC stable
- Level shifter bidirectionnel obligatoire sur TX/RX des capteurs 5V

II.7 Protocole MQTT — Conception du flux de données

II.7.1 Modèle Publish/Subscribe

Le firmware publie les mesures sur un broker **Mosquitto** (Eclipse, open source) via le protocole MQTT 3.1.1. Le modèle Publish/Subscribe découple la station ESP32 du backend : la station publie sans connaître les abonnés, ce qui permet d'ajouter de nouveaux consommateurs (tableau de bord secondaire, système d'archivage) sans modifier le firmware.



II.7.2 Conception de la structure des topics

Le schéma de topic est conçu pour être **hiérarchique et extensible** :

```
emissions / <NODE_ZONE> / <sensorType>
```

Exemples pour la station Station-Sfax-01 en Zone-A :

Topic MQTT	Polluant	Capteur physique
emissions/Zone-A/CO2	CO ₂	MH-Z19B
emissions/Zone-A/SO2	SO ₂	MQ-136
emissions/Zone-A/NOX	NOx	MQ-131
emissions/Zone-A/PM25	PM2.5	SDS011
emissions/Zone-A/PM10	PM10	SDS011
emissions/Zone-A/COV	TVOC	SGP30
emissions/Zone-A/TEMPERATURE	Température	DHT22
emissions/Zone-A/HUMIDITY	Humidité	DHT22

Le segment `<NODE_ZONE>` correspond au code de la zone MongoDB (`Zone.code`). Le backend résout le mapping `topic` → `Zone` → `SensorNode` → `Sensor` dans la base de données lors de l'ingestion.

II.7.3 Niveau de qualité de service (QoS)

Le firmware publie avec **QoS 1** (At-least-once) : le broker envoie un acquittement PUBACK à l'ESP32. En l'absence d'ACK, le message est retransmis. Ce choix garantit qu'aucune mesure de dépassement ne sera perdue lors d'un instant de congestion réseau, au prix d'éventuels doublons. Ces doublons sont gérés côté backend par déduplication temporelle.

QoS	Garantie	Raison du choix
QoS 0	Fire and forget — perte possible	Non retenu — données réglementaires
QoS 1	At-least-once — ACK broker	Retenu
QoS 2	Exactly-once — 4 messages handshake	Surcoût inutile, microcontrôleur limité

II.7.4 Structure du payload JSON

Chaque publication MQTT est un document JSON de ~380 octets. Le buffer MQTT est configuré à **512 octets** (`mqtt.setBufferSize(512)`) pour absorber cette taille avec marge.

```
{
  "sensorType": "SO2",
  "model":      "MQ-136",
  "zone":       "Zone-A",
  "nodeName":   "Station-Sfax-01",
  "value":      12.01,
  "rawValue":   0.336,
  "unit":       "mg/Nm³",
  "level":      "normal",
  "timestamp":  "2026-06-04T08:32:45Z",
  "isValid":    true,
  "rssi":       -62,
  "battery":    null
}
```

Sémantique des champs :

Champ	Type	Description
<code>sensorType</code>	String	Code polluant (CO2, SO2, NOX, PM25...) — résolu dans MongoDB
<code>model</code>	String	Référence hardware du capteur
<code>value</code>	Number	Valeur traitée post-conversion (mg/Nm³, ppm, µg/m³)
<code>rawValue</code>	Number	Valeur brute : tension ADC en V (MQ), ppb (SGP30) — diagnostic
<code>level</code>	String	Niveau indicatif local (<code>normal/warning/high/critical</code>)
<code>isValid</code>	Boolean	<code>false</code> si lecture hors plage — le backend stocke sans déclencher d'alerte
<code>rssi</code>	Number	Qualité du lien WiFi en dBm — diagnostic réseau
<code>battery</code>	null	Confirme l'alimentation permanente sur secteur (pas de gestion batterie)

II.7.5 Fréquences de publication

Les intervalles sont définis dans `config.h` et justifiés par les caractéristiques physiques de chaque capteur :

Famille	Intervalle	Justification
CO ₂ (MH-Z19B)	10 s	Réponse UART rapide, ABC sur longue période
T°/HR (DHT22)	10 s	Limite datasheet : 0,5 Hz maximum
PM (SDS011)	15 s	Cycle laser + trame UART + stabilisation ventilateur
Gaz MQ + COV SGP30	30 s	Moyennage 16 × 5 ms ADC + T90 ≤ 30 s pour stabilisation MOS

Estimation de la charge MQTT : 8 topics × ~380 octets × (3 publ./30 s + 2 publ./15 s + 4 publ./10 s) ≈ **12**

messages/minute → ~4,6 KB/min de trafic montant. Cette charge est très largement compatible avec une connexion WiFi industrielle standard (1 Mbit/s en 802.11b).

II.8 Architecture logicielle du firmware

II.8.1 Séparation des responsabilités

Le firmware est structuré en deux couches distinctes :

```
main.cpp - couche transport et orchestration
├── Connexion WiFi et MQTT
├── Boucle non bloquante (millis())
├── Construction payload JSON (ArduinoJson)
└── Publication MQTT (PubSubClient)
```

```
sensors.cpp - couche acquisition bas niveau
├── readCo2() → MH-Z19B, UART, MHZ19 lib
├── readParticulates() → SDS011, UART, SdsDustSensor
├── readSo2() → MQ-136, ADC, calcul Rs/R0
├── readNox() → MQ-131, ADC, calcul Rs/R0
├── readCov() → SGP30, I²C, Adafruit lib
└── readEnvironment() → DHT22, 1-wire, DHT lib
```

II.8.2 Boucle principale non bloquante

Le firmware adopte une architecture de planification basée sur `millis()` sans aucun `delay()` bloquant dans le chemin critique. Cela garantit que les reconnexion réseau ne retardent pas les acquisitions, et que le watchdog interne de l'ESP32 n'est pas déclenché.

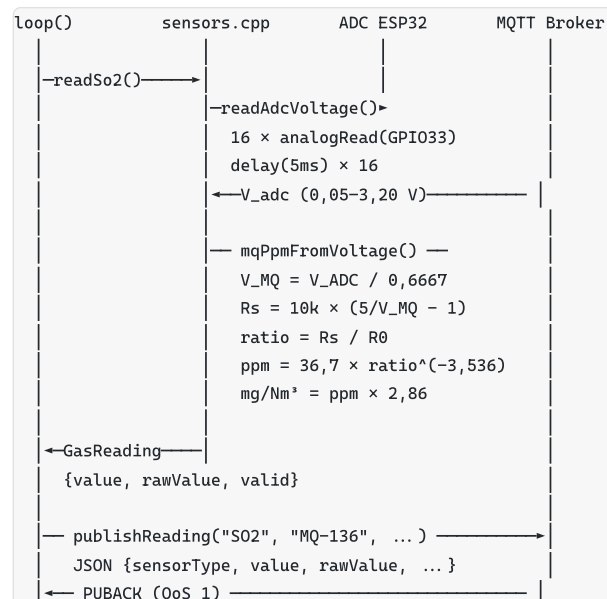
Conception de la boucle loop() :

```
void loop() {
  // Maintenance réseau
  if (WiFi déconnecté) → reconnexion
  if (MQTT déconnecté) → reconnexion
  mqtt.loop() ← keep-alive + callbacks

  // Planification capteurs
  if (now - lastCo2 ≥ 10 000 ms) → publishCo2()
  if (now - lastPm ≥ 15 000 ms) → publishParticulates()
  if (now - lastGas ≥ 30 000 ms) → publishGases()
  if (now - lastEnv ≥ 10 000 ms) → publishEnvironment()

  delay(50) ← cède au watchdog et au stack WiFi
}
```

II.8.3 Diagramme de séquence — Cycle de publication SO₂



II.8.4 Stratégie de reconnexion

Le firmware implémente une reconnexion automatique sur deux niveaux :

- **WiFi** : la boucle `loop()` appelle `connectWiFi()` à chaque itération si `WiFi.status() != WL_CONNECTED`. La reconnexion est bloquante uniquement le temps d'obtenir l'IP.
- **MQTT** : la boucle appelle `connectMqtt()` si `!mqtt.connected()`. Entre chaque tentative, un délai de 3 secondes permet au broker de se remettre en état sans surcharger le réseau.

La synchronisation **NTP** est effectuée dans `setup()` via `configTime(0, 0, "pool.ntp.org", "time.nist.gov")` pour garantir des timestamps ISO 8601 UTC corrects dans les payloads. En cas d'échec NTP, la fonction `isoTimestamp()` retourne une chaîne vide ; le backend utilise `new Date()` comme valeur de repli.

II.9 Tableau de synthèse — Conception complète de la station

Paramètre	Valeur retenue	Justification
Carte	ESP32 DevKit V1 (WROOM-32)	WiFi natif, 3 UART, ADC1 WiFi-safe, coût 15 DT
Alimentation	Bloc 5V/3A → VIN ESP32 + bus 5V direct	Bilan courant 810 mA nominal, marge ×2,3
Rail 3,3V	LDO interne AMS1117 (broche 3V3 ESP32)	Charge 3,3V ≈ 130 mA << 800 mA max — LM2596 externe inutile
Protection	Fusible 1A + diode 1N4007	Surintensité + inversion polarité
Découplage	100 nF céramique par IC au plus près	Filtrage transitoires HF que le LDO ne peut pas atténuer
CO ₂	MH-Z19B (NDIR) — Serial2 + level shifter	Sélectivité CO ₂ , ABC long terme
SO ₂	MQ-136 (MOS) — ADC GPIO33, div. 10k/20k, NPN GPIO26	Disponibilité locale, 8 DT
NO _x	MQ-131 (MOS) — ADC GPIO34, div. 10k/20k, NPN GPIO27	Disponibilité locale, 10 DT
COV	SGP30 (I ² C 0x58) — SDA/SCL pull-up 4,7kΩ	Pas de circuit analogique, 3,3V natif
PM2.5/PM10	SDS011 (laser) — Serial1, mode query	Double mesure sur 1 UART
T°/HR	DHT22 (1-wire) — GPIO4, pull-up 10kΩ	Disponibilité immédiate, interface simple
Filtrage ADC	Moyennage 16 échantillons × 5 ms	Réduction bruit ÷4, résolution +2 bits
Protocole	MQTT 3.1.1, QoS 1, Mosquitto	Livraison garantie, découplage producteur/consommateur
Topics	<code>emissions/<zone>/<type></code>	Hierarchique, extensible, wildcard # backend
Payload	JSON ~380 octets, buffer 512 octets	Compatible simulateur Node.js, tous champs requis
Fréquences	CO ₂ /T°HR : 10 s · PM : 15 s · Gaz : 30 s	Adaptées aux T90 et limites physiques de chaque capteur
Architecture firmware	Non bloquant millis(), séparation sensors/main	Robustesse reconnexion, watchdog safe